

called `__pycache__`. To tell Git to ignore this directory, make a special file called `.gitignore`—with a dot at the beginning of the filename and no file extension—and add the following line to it:

```
.gitignore __pycache__/
```

This file tells Git to ignore any file in the `__pycache__` directory. Using a `.gitignore` file will keep your project clutter-free and easier to work with.

You might need to modify your file browser's settings so hidden files (files whose names begin with a dot) will be shown. In Windows Explorer, check the box in the View menu labeled **Hidden Items**. On macOS, press ⌘-SHIFT-. (dot). On Linux, look for a setting labeled Show Hidden Files.

NOTE

If you're on macOS, add one more line to `.gitignore`. Add the name `.DS_Store`; these are hidden files that contain information about each directory on macOS, and they will clutter up your project if you don't add them to `.gitignore`.

Initializing a Repository

Now that you have a directory containing a Python file and a `.gitignore` file, you can initialize a Git repository. Open a terminal, navigate to the `git_practice` folder, and run the following command:

```
git_practice$ git init
Initialized empty Git repository in git_practice/.git/
git_practice$
```

The output shows that Git has initialized an empty repository in `git_practice`. A *repository* is the set of files in a program that Git is actively tracking. All the files Git uses to manage the repository are located in the hidden directory `.git`, which you won't need to work with at all. Just don't delete that directory, or you'll lose your project's history.

Checking the Status

Before doing anything else, let's look at the project's status:

```
git_practice$ git status
❶ On branch main
No commits yet

❷ Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore
    hello_git.py

❸ nothing added to commit but untracked files present (use "git add" to track)
git_practice$
```
